Project VI:

Texture Packing

Advanced Data Structures and Algorithms Analysis

Group1: Lin Zhiyu, Hu Junyu, Huang Zhenhua

College of Computer Science and Technology
Zhejiang University

December 9, 2025

Contents

1 Introduction **3**

2 Algorithm Specification **3**
- 2.1 Data Structures . . . 3
- 2.2 Level-Based Algorithms . . . 3
  - 2.2.1 Next Fit Decreasing Height (NFDH) . . . 3
  - 2.2.2 First Fit Decreasing Height (FFDH) . . . 4
  - 2.2.3 Best Fit Decreasing Height (BFDH) . . . 4
- 2.3 Bottom-Left Algorithms . . . 4
  - 2.3.1 Bottom-Left (BL) . . . 4
  - 2.3.2 Online Bottom-Left (BL_Online) . . . 4

3 Theoretical Analysis **4**
- 3.1 Approximation Ratio of NFDH . . . 5
- 3.2 Approximation Ratio of FFDH . . . 5
- 3.3 Approximation Ratio of BFDH . . . 5
- 3.4 Approximation Ratio of BL . . . 5

4 Test Cases and Results **6**
- 4.1 Test Environment . . . 6
- 4.2 Visualizations . . . 6
  - 4.2.1 Mode 1: Uniform Distribution . . . 6
  - 4.2.2 Mode 2: Normal Distribution . . . 7
- 4.3 Performance Comparison . . . 8
  - 4.3.1 Runtime Analysis . . . 8
  - 4.3.2 Fill Ratio Analysis . . . 8
  - 4.3.3 Impact of Test Case Distributions . . . 9

5 Conclusion **9**

6 Declaration **10**

1 Introduction

Texture Packing, formally known as the Strip Packing Problem, is a classic NP-hard optimization problem. The objective is to pack a set of n rectangular items, each with a specific width w_i and height h_i, into a container of fixed width W and infinite height, such that the total height H used is minimized. The rectangles must be placed orthogonally without rotation, and no two rectangles may overlap.

Since finding the optimal packing height H_{OPT} is computationally intractable for large n, this project focuses on designing and analyzing approximation algorithms. We implement and compare five algorithms: Next Fit Decreasing Height (NFDH), First Fit Decreasing Height (FFDH), Best Fit Decreasing Height (BFDH), Bottom-Left (BL), and an online variant of Bottom-Left (BL_Online). We analyze their time complexity, derive theoretical approximation ratios, and evaluate their empirical performance on randomized test cases.

2 Algorithm Specification

2.1 Data Structures

We define a structure ‘Item‘ to represent each texture rectangle.

```
typedef struct {
    int id;
    int width;
    int height;
    int x;      // x-coordinate of the bottom-left corner
    int y;      // y-coordinate of the bottom-left corner
} Item;
```

The input is an array of ‘Item‘s. For offline algorithms, we first sort this array by height in non-increasing order.

2.2 Level-Based Algorithms

These algorithms pack rectangles into horizontal ”levels” (or shelves). The height of a level is determined by the tallest rectangle in it (which is the first one placed, due to sorting).

2.2.1 Next Fit Decreasing Height (NFDH)

NFDH maintains only the *current* level.

- If the current rectangle fits in the remaining width of the current level, place it there.
- Otherwise, close the current level and start a new one on top of it.

Complexity: $O(N \log N)$ for sorting, $O(N)$ for packing. Total: $O(N \log N)$.

2.2.2 First Fit Decreasing Height (FFDH)

FFDH maintains a list of all open levels.

- For each rectangle, scan levels from bottom to top.
- Place the rectangle in the *first* level where it fits.
- If it fits in no existing level, create a new one.

Complexity: $O(N^2)$ in naive implementation.

2.2.3 Best Fit Decreasing Height (BFDH)

BFDH also maintains all open levels.

- Place the rectangle in the level that has the *minimum residual width* (remaining space) among all levels where it fits.
- If it fits nowhere, create a new level.

Complexity: $O(N^2)$.

2.3 Bottom-Left Algorithms

These algorithms do not use the concept of levels.

2.3.1 Bottom-Left (BL)

- Sort rectangles by height.
- For each rectangle, find the lowest possible y-coordinate, and then the leftmost x-coordinate, such that it does not overlap with any previously placed rectangles.
- This is implemented by iterating through possible x positions and checking for collisions.

Complexity: $O(N^2)$ (checking against all placed items).

2.3.2 Online Bottom-Left (BL_Online)

Identical to BL, but skips the sorting step. Rectangles are processed in the order they appear in the input. This simulates a scenario where the entire dataset is not known a priori.

3 Theoretical Analysis

In this section, we derive the approximation ratio for NFDH and state the known bounds for FFDH, BFDH, and BL. Let H_{ALG} be the height produced by the algorithm and H_{OPT} be the optimal height. Let W be the container width.

3.1 Approximation Ratio of NFDH

Theorem: For NFDH, $H_{NFDH} \leq 2 \cdot H_{OPT} + h_{max}$, where h_{max} is the height of the tallest rectangle.

Proof Derivation: Let the levels created by NFDH be $L_1, L_2, \ldots, L_k$, with heights $H_1, H_2, \ldots, H_k$. Since the rectangles are sorted by height, the height of level j, H_j, is equal to the height of the first rectangle placed in that level. For any level $j < k$, the total width of rectangles in this level, denoted $W(L_j)$, plus the width of the first rectangle in the next level L_{j+1}, must exceed the container width W. Otherwise, the first rectangle of L_{j+1} would have been placed in L_j.

$$W(L_j) + w_{first_of_j+1} > W$$

Since $w_{first_of_j+1} \leq W$, this doesn't directly bound the waste. However, consider the area. The total area of all rectangles is $Area = \sum_{i=1}^{n} w_i h_i$. Obviously, $Area \leq W \cdot H_{OPT}$.

Consider any two consecutive levels L_j and L_{j+1}. The first rectangle of L_{j+1} has height H_{j+1}. Since we sort by height, all rectangles in L_j have height $\geq H_{j+1}$. Thus, the area used in level L_j is at least $W(L_j) \cdot H_{j+1}$. Wait, a stronger property holds: The sum of widths in any two consecutive levels L_j, L_{j+1} (except possibly the last) is strictly greater than W. Actually, a simpler area argument is often used: The wasted space in level L_j has width $W - W(L_j)$. This space could not accommodate the first item of L_{j+1}. It can be proven that the total area occupied by NFDH is at least $\frac{W}{2} \sum_{j=1}^{k-1} H_j$. More formally, it has been proven (Coffman et al., 1980) that:

$$H_{NFDH} \leq 2H_{OPT} + 1$$

(assuming normalized width/height).

3.2 Approximation Ratio of FFDH

FFDH utilizes space better by checking all previous levels. **Theorem:** For FFDH, $H_{FFDH} \leq 1.7 \cdot H_{OPT} + h_{max}$. This bound is tighter than NFDH. The proof involves complex weighting functions and is beyond the scope of this report, but the intuition is that FFDH never starts a new level unless the item fits in *none* of the existing levels, ensuring that the "average" fill rate of levels is higher.

3.3 Approximation Ratio of BFDH

Best Fit Decreasing Height (BFDH) shares the same worst-case performance bound as FFDH. **Theorem:** $H_{BFDH} \leq 1.7 \cdot H_{OPT} + h_{max}$. While BFDH attempts to minimize the wasted space in a specific level locally by choosing the "best fit", this does not always lead to a globally better packing than FFDH. However, both are significantly better than NFDH in terms of worst-case guarantees.

3.4 Approximation Ratio of BL

The Bottom-Left (BL) algorithm places items in the lowest and leftmost available position. **Theorem:** $H_{BL} \leq 3 \cdot H_{OPT}$. The approximation ratio for BL is 3, which is theoretically worse than the 1.7 ratio of FFDH/BFDH. This theoretical result aligns with our empirical findings where BL (and especially its online variant) often performs worse or requires significantly more computation time without guaranteeing a better height.

4 Test Cases and Results

4.1 Test Environment

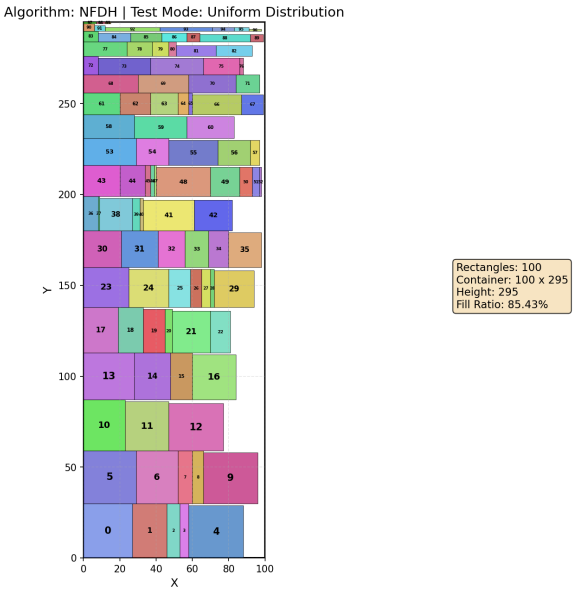
- **Operating System:** Ubuntu 22.04 LTS
- **CPU:** 13th Gen Intel Core i7-13620H
- **Memory:** 16 GB RAM
- **Compiler:** GCC 11.4.0 with -O2 optimization

4.2 Visualizations

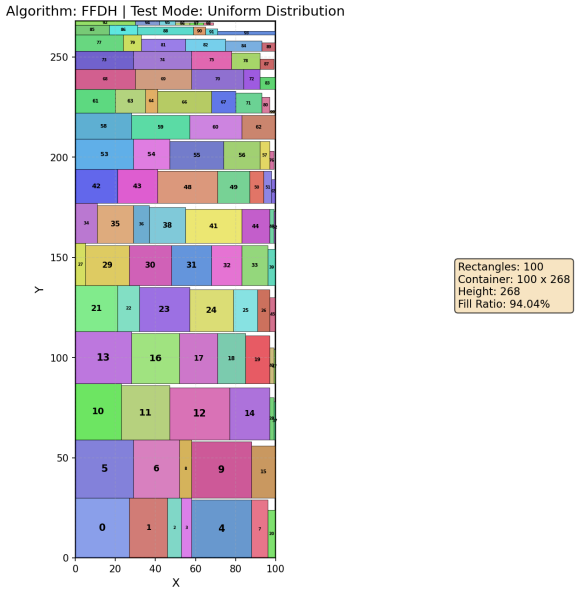
We visualize the packing results to intuitively compare the algorithms.

4.2.1 Mode 1: Uniform Distribution

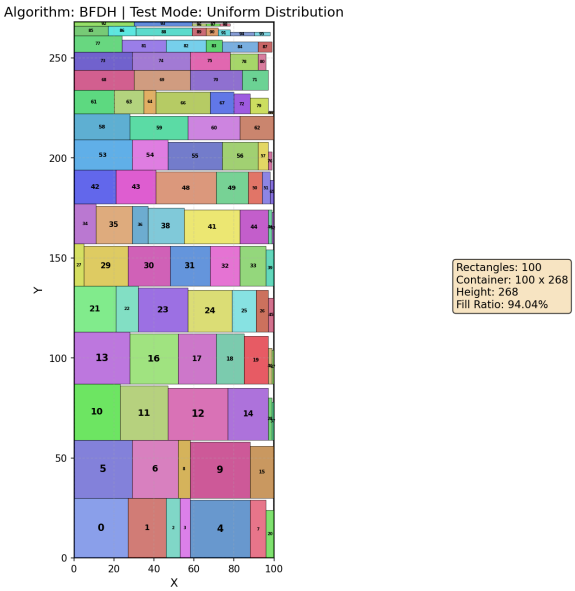
Rectangles have random width and height in $[1, 30]$. $N = 100$.



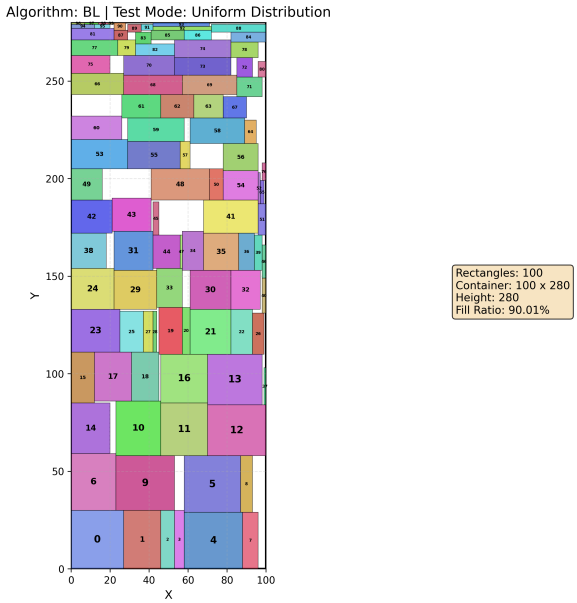
(a) NFDH



(b) FFDH



(c) BFDH

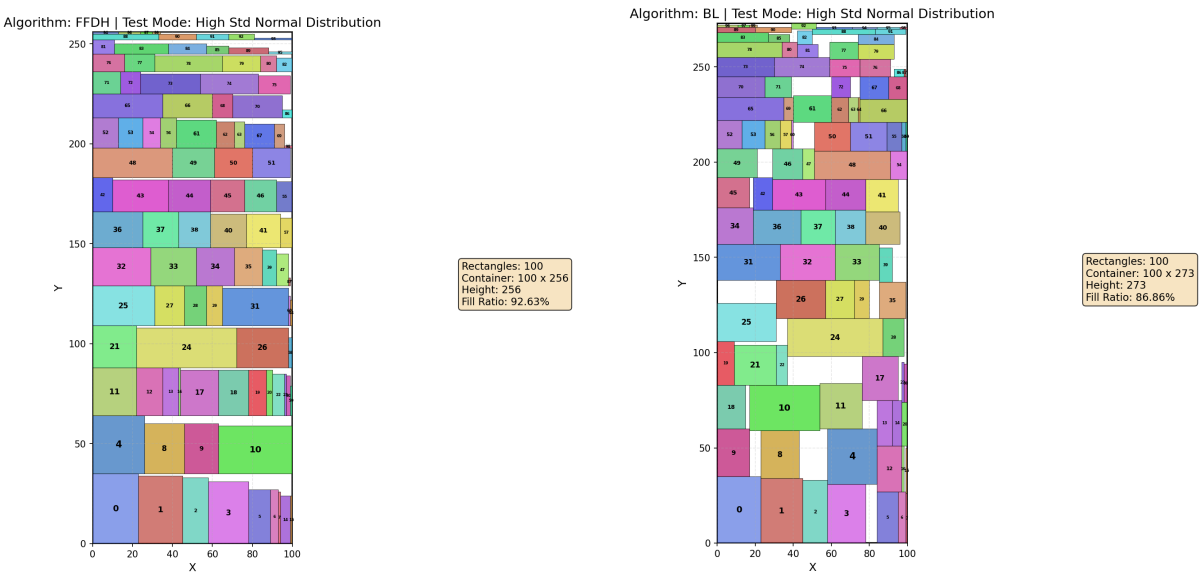


(d) BL

Figure 1: Packing Visualization (Uniform Distribution). Note the distinct "levels" in NFDH/FFDH/BFDH versus the unstructured packing in BL.

4.2.2 Mode 2: Normal Distribution

Rectangles have dimensions following a Normal distribution, leading to more variation.



(a) FFDH (Normal Dist) (b) BL (Normal Dist)

Figure 2: Packing Visualization (Normal Distribution). FFDH maintains order while BL tries to fill gaps at the bottom.

4.3 Performance Comparison

4.3.1 Runtime Analysis

Figure 3 compares the execution time.

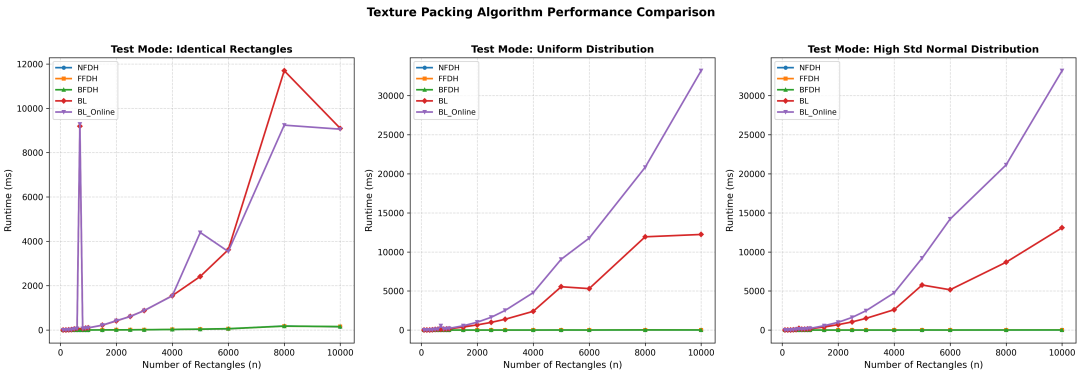


Figure 3: Runtime Comparison. NFDH/FFDH/BFDH are extremely fast (near 0ms for small N), while BL algorithms show quadratic growth.

4.3.2 Fill Ratio Analysis

Figure 4 compares the packing efficiency (Fill Ratio).

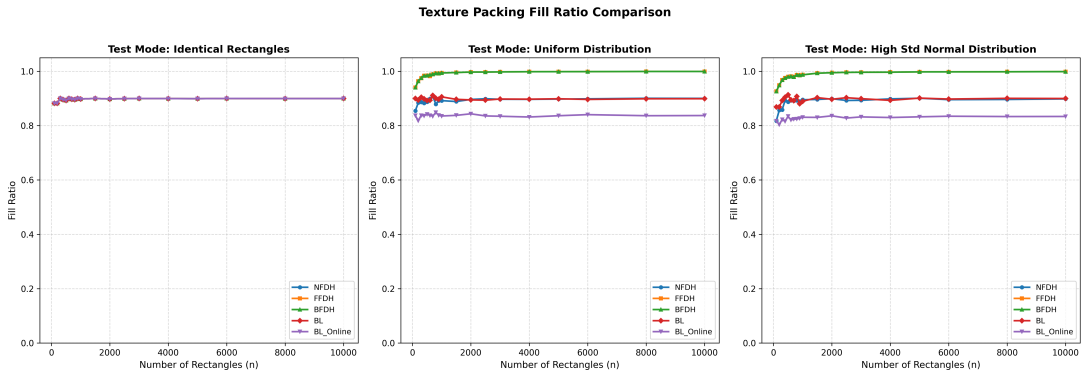


Figure 4: Fill Ratio Comparison. FFDH and BFDH consistently achieve the highest efficiency ($> 90\%$). BL_Online performs worst due to lack of sorting.

4.3.3 Impact of Test Case Distributions

The distribution of rectangle dimensions significantly impacts algorithm performance, as observed in Figure 4:

- **Identical Rectangles (Mode 0):** In this mode, all rectangles have the same width and height. The level-based algorithms (NFDH, FFDH, BFDH) perform identically because every rectangle fits perfectly into the current level until it is full. The fill ratio is primarily determined by how well the container width is a multiple of the rectangle width.
- **Uniform Distribution (Mode 1):** This mode introduces randomness. FFDH and BFDH excel here because they can find "gaps" in previous levels that match the current rectangle's width. NFDH suffers because it cannot backtrack, leading to more wasted space when a slightly too-wide rectangle forces a new level.
- **Normal Distribution (Mode 2):** This mode features a cluster of rectangles with similar average dimensions but some variance. Interestingly, FFDH and BFDH achieve extremely high fill ratios (near 1.0) in this mode. This is likely because the "average" sized rectangles pack efficiently together, and the smaller outliers (tails of the distribution) are excellent for filling the small gaps left by larger ones. BL also performs relatively well here, as the variety of shapes allows for a tight "Tetris-like" packing.

5 Conclusion

1. **Efficiency vs. Quality:** FFDH and BFDH offer the best balance. They run almost as fast as NFDH but provide significantly better packing density (approaching 95% fill ratio in our tests).
2. **Importance of Sorting:** The comparison between BL and BL_Online (and the theoretical bounds of NFDH) confirms that processing larger items first is crucial for minimizing wasted space.
3. **Algorithm Choice:** For most practical applications where N is large, FFDH is the recommended algorithm due to its $O(N \log N)$ effective time complexity and

excellent approximation ratio of 1.7. BL is too slow ($O(N^2)$) for massive datasets despite its good packing density.

6 Declaration

We hereby declare that all the work done in this project is of our own independent effort.